FORMADORES { IT }

Kubernetes

9. Arquitectura & on-premise

Índice

Arquitectura de Kubernetes

Control Plane

Workers

Networking

Despliegue on-premise

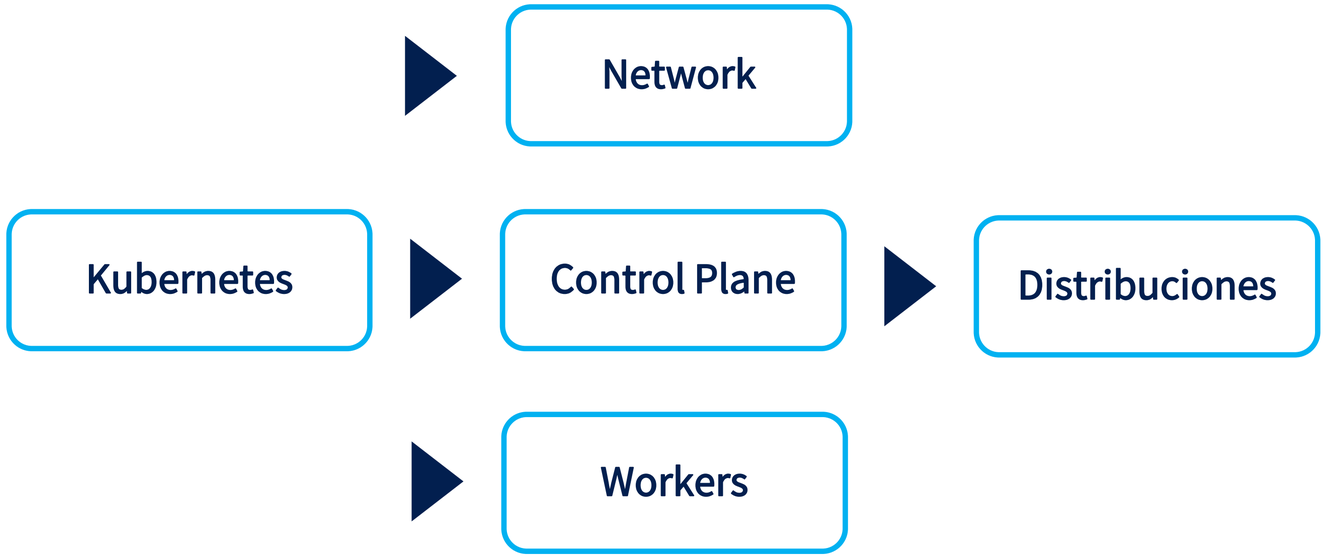
Distribuciones

Entornos de pruebas / local

Objetivos

- Ver en profundidad la arquitectura de Kubernetes
- Conocer la función que realiza cada pieza
- Aprender a configurar la red para un clúster
- Ver distintas opciones de despliegue

Mapa Conceptual

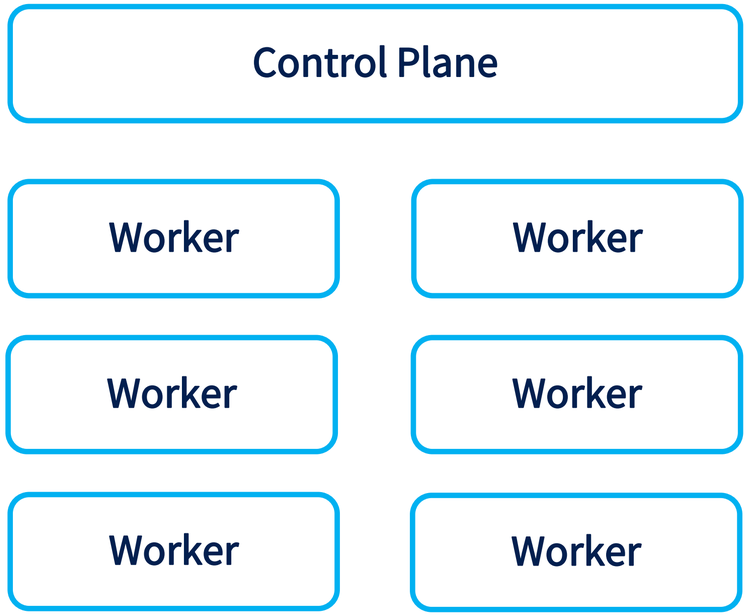


Arquitectura

Básicamente Kubernetes tiene:

1. Panel de control o **Control Plane** . Son los servicios que controlan el clúster
2. Nodos o **workers**. Son las máquinas en las que corren los contenedores

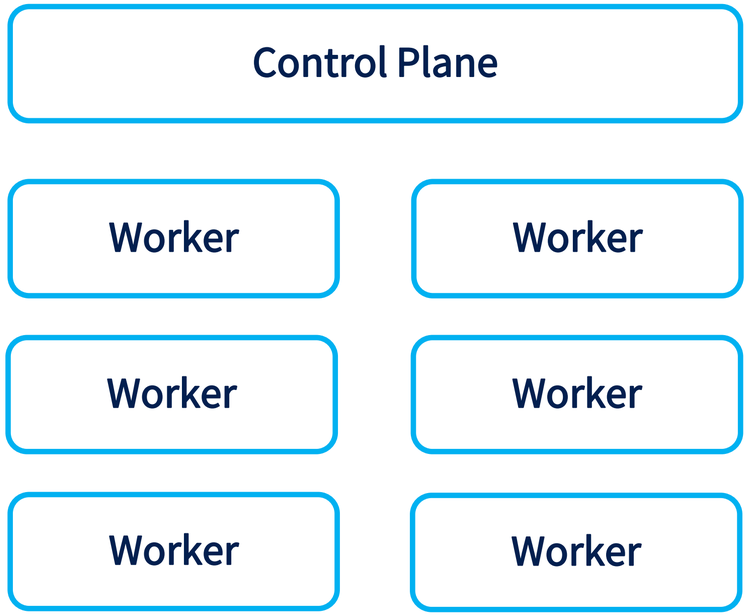
Típica arquitectura *master - slave*



¿Por qué esta arquitectura?

Separar las cargas de trabajo de la administración del clúster tiene muchas ventajas:

- En caso de cualquier error en el Control Plane, las aplicaciones continuarían funcionando como en el último estado válido.
- Y al revés, un error en un worker no afecta a los demás y al Control Plane, que siguen funcionando sin él.
- Seguridad: Nadie accede directamente a las cargas de trabajo. Estas no están expuestas por ningún motivo. Salvo los puertos que son abiertos a Internet para acceder a las apps.
- Toda interacción se hace a través del api-server. Esto facilita la auditoría de acciones, el *debugging*, y la gestión de permisos.
- Cada pieza se puede actualizar independientemente y sin causar caídas.



Control Plane

Donde ocurre la magia de Kubernetes

Se compone de 3 servicios:

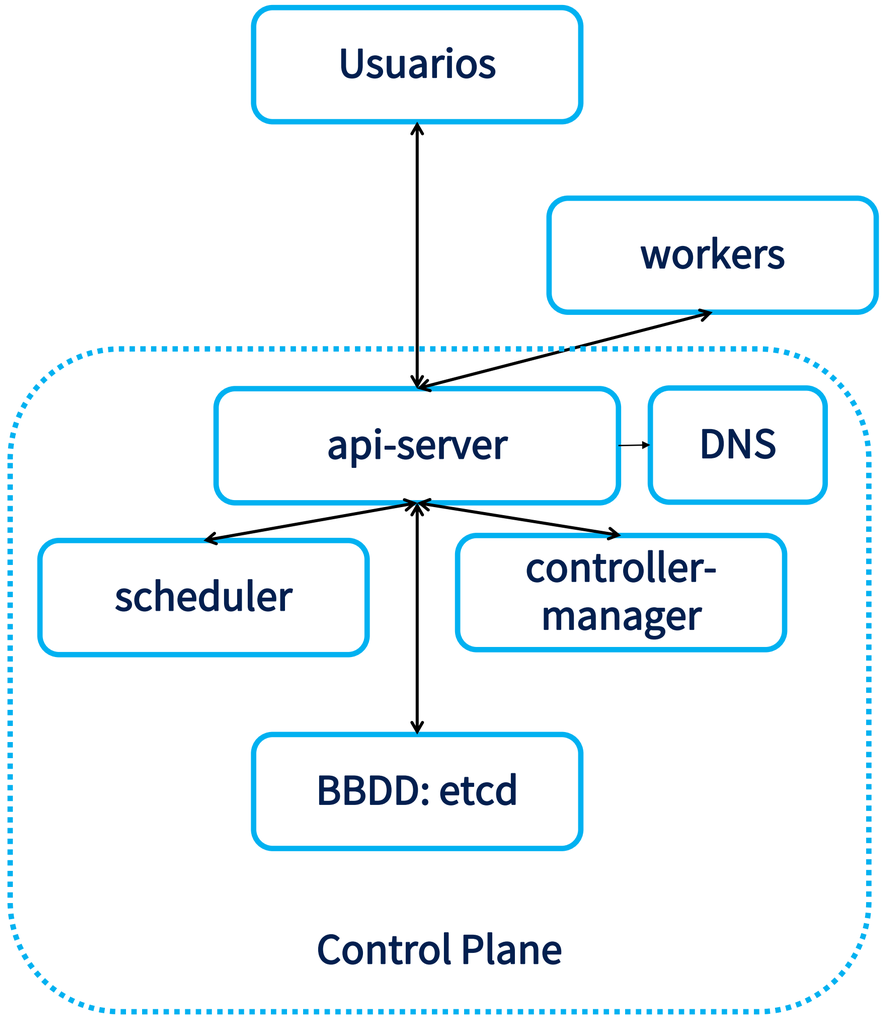
- **api-server**
- **controller-manager**
- **scheduler**

Una base de datos:

- **etcd**

E incluye también el servidor DNS:

- **CoreDNS / kube-dns**



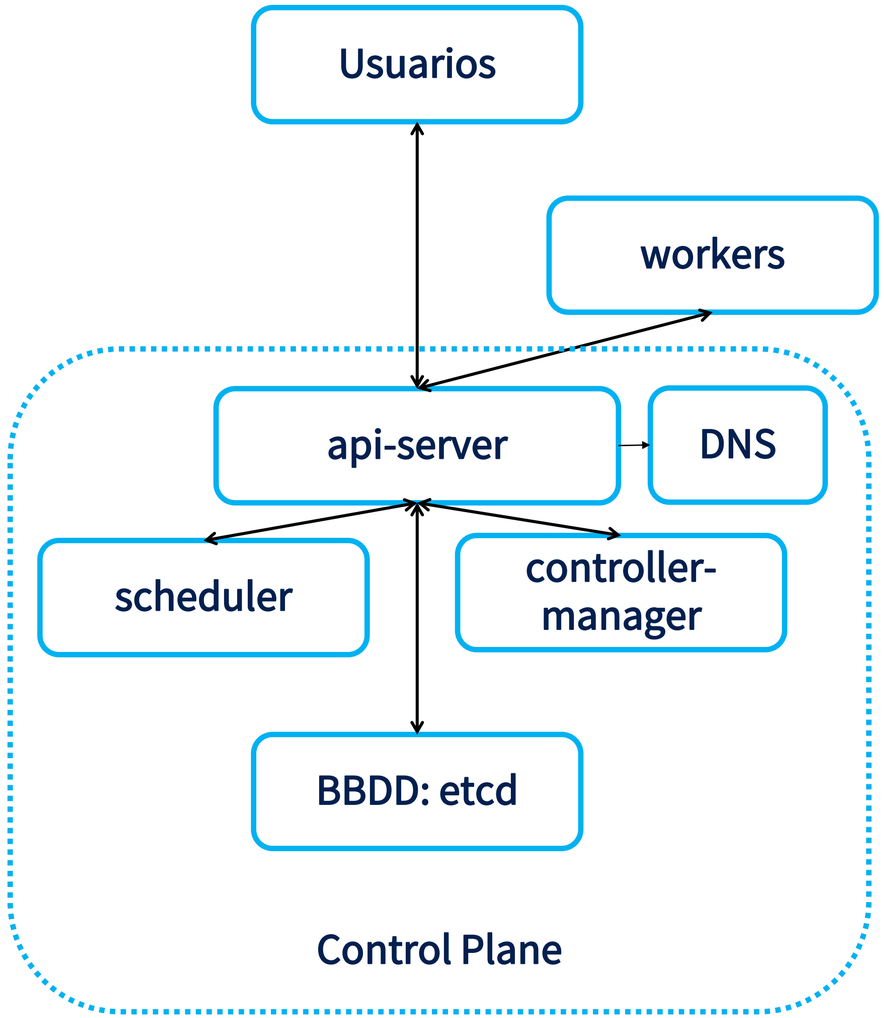
Control Plane

Dependiendo del clúster puede haber más piezas cuya función es coordinarse con sistemas cloud o de almacenamiento, pero estas 5 son imprescindibles en todos ellos.

etcd

Es la base de datos de Kubernetes, donde se almacenan todos los objetos que existen en el clúster. Siempre que creemos un objeto, se actualice automáticamente su estado, lo eliminemos etc, esos cambios se almacenaran en etcd.

Es básicamente el almacen de objetos del clúster.

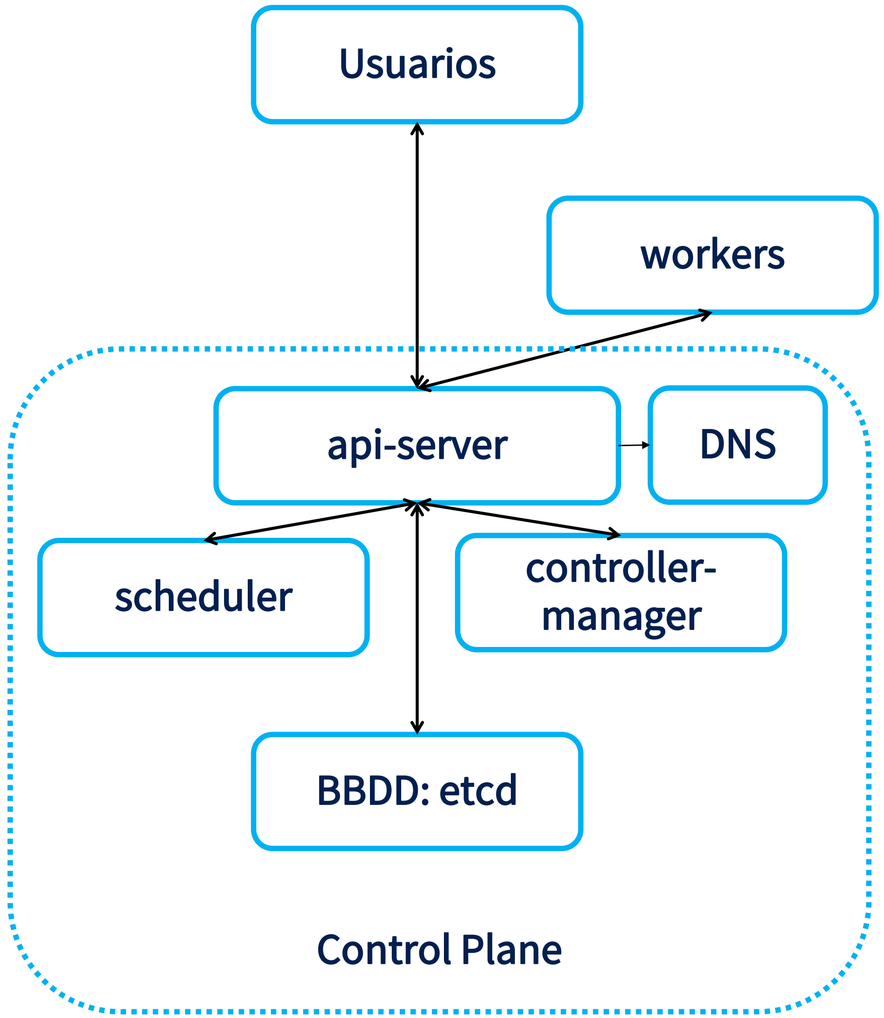


Control Plane

API Server

Es el punto de entrada para todas las comunicaciones entre el control plane y cualquier otro sistema (nodos, cloud, sistemas de almacenamiento, usuarios como nosotros, etc.). Ninguna otra pieza recibe datos del exterior.

El API Server procesa todos los comandos y actualizaciones de estado, y se comunica a su vez con las demás piezas para determinar qué acciones concretas son necesarias en el clúster. También almacena todos los datos en la base de datos etcd.



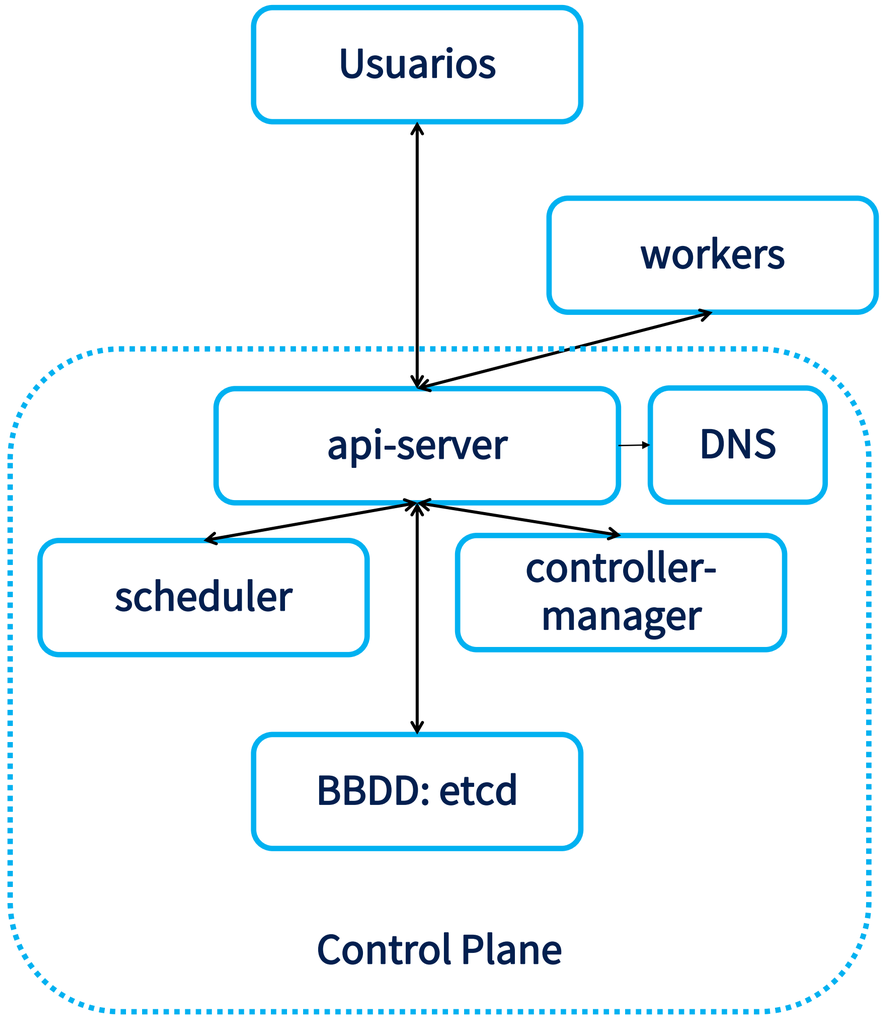
Control Plane

Controller Manager

Un controller en general es un bucle infinito que controla el estado de un sistema. En Kubernetes el controller manager se encarga exactamente de eso, de vigilar que el sistema esté en el estado que se desea, y de ejecutar las acciones necesarias para que así sea.

Por ejemplo, cuando se pierde la conexión con un nodo, el controller manager observa que sus pods han dejado de existir, y levanta pods nuevos en otros nodos.

Este es sólo un ejemplo, el controller manager se compone de muchas piezas distintas que vigilan el estado de todos los elementos del clúster.

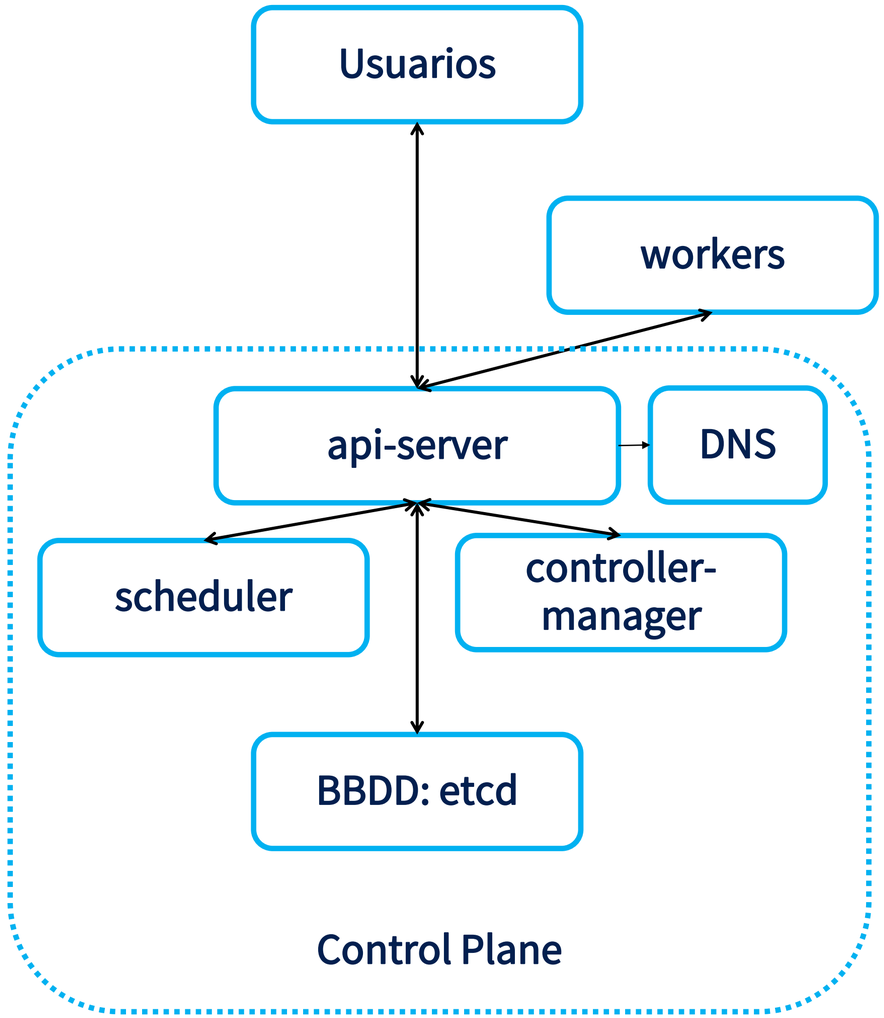


Control Plane

Scheduler

Es el encargado de pensar en qué nodo se ejecutará cada pod. Cuando creamos un pod el primer estado en el que se encontrará es Pending.

En ese estado aún no tiene un nodo asignado; el scheduler está procesando en qué nodos puede ejecutarse (esto depende de muchas cosas como restricciones, requests de recursos, volúmenes persistentes, etc), y de elegir el más conveniente entre ellos para que siempre haya un equilibrio.

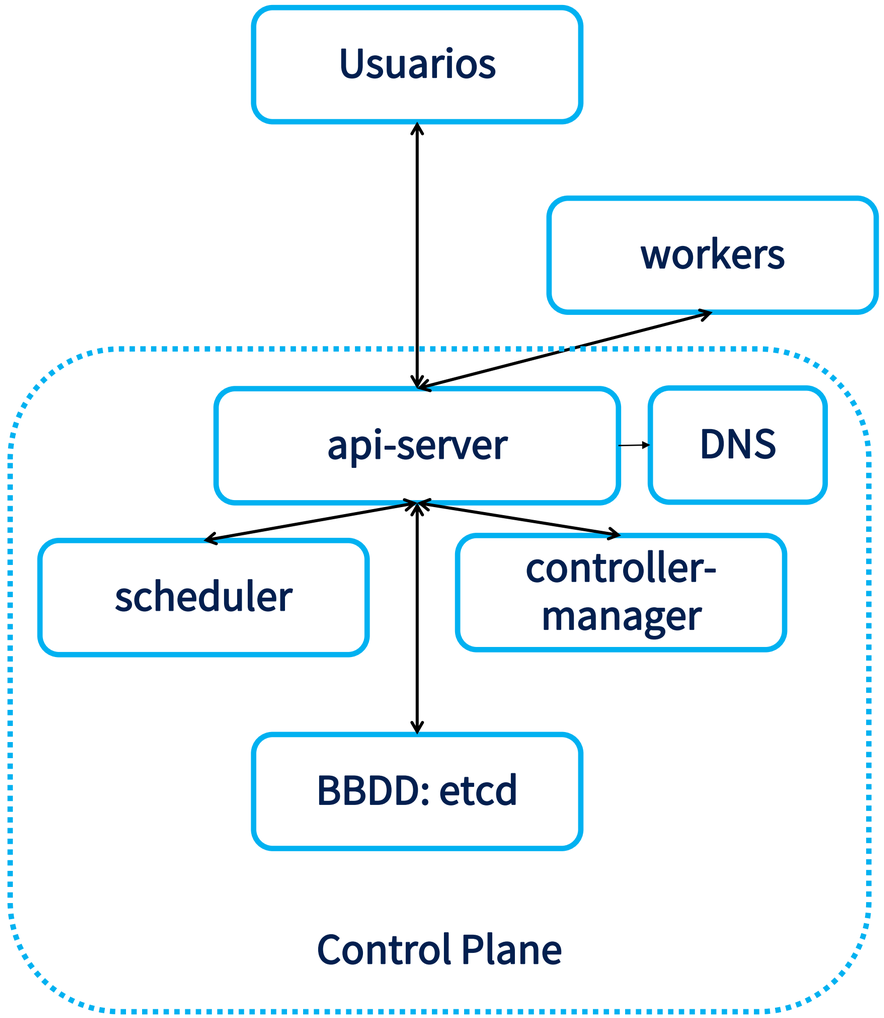


Control Plane

CoreDNS o kube-dns

Esta pieza ya la conocemos, es el servidor de nombres o DNS interno del clúster de Kubernetes. Crea direcciones para todos los services y pods del clúster.

Utiliza por defecto el dominio cluster.local para estas direcciones. Además, todos los pods del clúster por defecto están configurados para utilizar este servicio DNS automáticamente. Es imprescindible su presencia en el clúster, sino los componentes no pueden comunicarse entre sí ya que casi nada tiene IP fija.



Workers

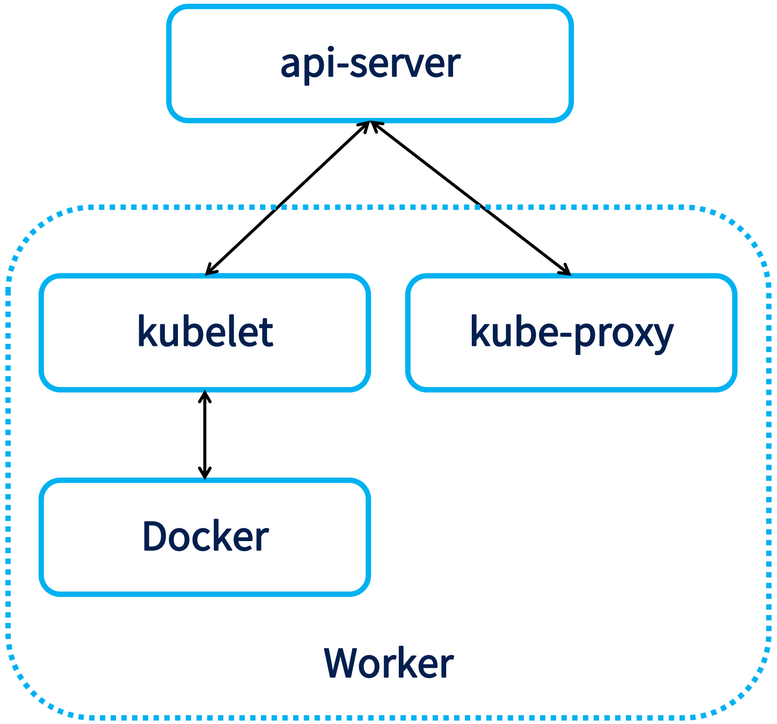
Los Workers o nodos de Kubernetes son la fuerza bruta. Son los que tienen los recursos de CPU, memoria y disco para ejecutar los contenedores. No realizan ninguna función extra, toda la gestión de recursos, salud de los sistemas, etc, se realiza en el control plane. Los worker sólo reciben y ejecutan sus comandos.

Siempre tienen dos agentes:

1. **kubelet**
2. **kube-proxy**

El Control Plane se encarga de decidir qué pods se ejecutan en cada worker, y manda las instrucciones necesarias a kubelet y kube-proxy.

En ningún momento es necesario comunicarse directamente con un worker. Son sólo recursos de ejecución dentro del clúster. El Control Plane ofrece todas las posibilidades para administrar nuestras aplicaciones a través de él.



Workers

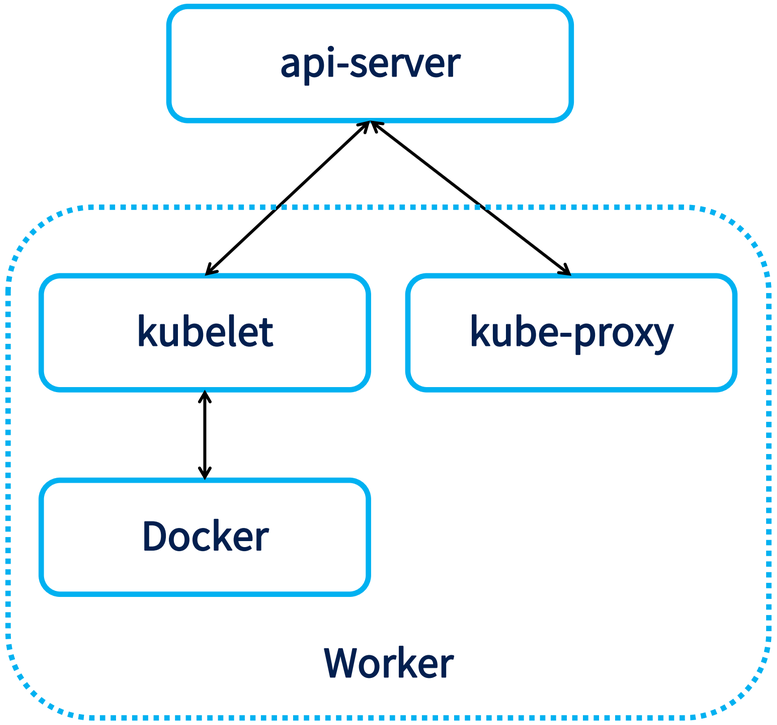
La pieza clave de un worker de Kubernetes es **Kubelet**.

Kubelet es el agente que se ejecuta en cada nodo. Lo primero que hace es registrarse en el API Server para que éste lo registre en el clúster como un nuevo nodo y comience a enviarle ordenes.

Kubelet sólo entiende de los objetos Pod. Todos los demás objetos no son tarea suya, sino que Kubelet sólo recibe los datos de los pod que debe ejecutar.

Una vez recibe estos datos, se comunica con el runtime de contenedores (en general, con Docker), para que éste levante los contenedores. Se encarga de pasar a Docker los parámetros correctos como variables de entorno, crear y montar todos los volúmenes, etc.

También los monitoriza y notifica al API Server con estos datos.

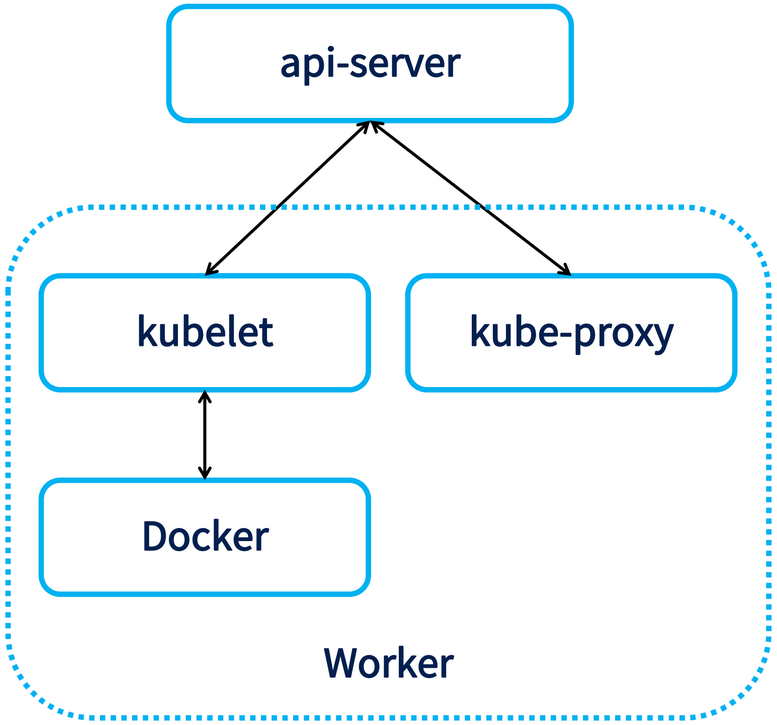


Workers

La otra pieza es la que ya conocemos: **kube-proxy**

kube-proxy observa a todos los services del cluster y crea reglas en el firewall iptables del nodo para implementar las IPs privadas de los servicios y los puertos de los NodePort.

Normalmente se despliega como un DaemonSet en el namespace kube-system.



Networking

El último aspecto a tener en cuenta para desplegar tu propio clúster de Kubernetes es el networking. Kubernetes tiene unos requisitos que tienen que cumplir todos los nodos del clúster.

EL principal es permitir que todos **los contenedores de todos los nodos se puedan comunicar entre sí con sus propias IPs**. Es decir, sin convertir estas IPs mediante NAT.

Normalmente cuando levantas contenedores en una máquina, estos obtienen una IP interna para esa máquina, pero al comunicarse con el exterior no lo hacen con esa IP interna, sino que utilizan la IP de la propia máquina. Kubernetes necesita que esas IPs internas se comuniquen entre sí.

Networking

Por suerte no necesitamos configurar routers y redes para conseguir esto. Existen multitud de plugins de red que crean esta red virtual entre todos los nodos de Kubernetes y que podemos desplegar en todos los nodos como cualquier otro pod. La práctica totalidad de implementaciones de Kubernetes que existen utilizan estos plugins de red para gestionar la comunicación entre contenedores.

Algunos de los más utilizados son:

- **Flannel**
- **Weave** (es el que se utiliza en el entorno de prueba de Katacoda, puedes ver sus pods en el namespace kube-system).
- Cilium
- Calico
- Y cada uno de los grandes cloud tiene su propio plugin que utiliza sus servicios de red

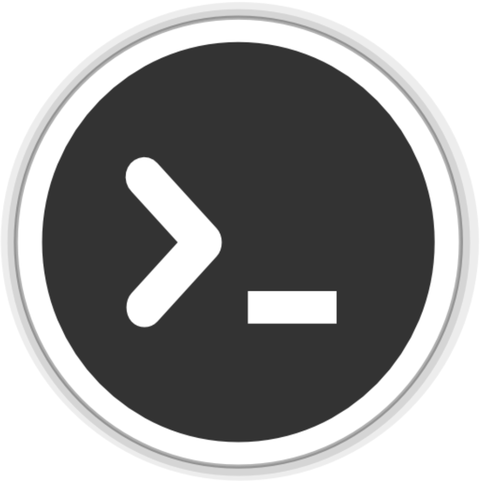
Exploramos estos componentes

En el entorno de Katacoda, los servicios del Control Plane se ejecutan como pods dentro del propio clúster. Así es, Kubernetes puede ‘ejecutarse a sí mismo’.

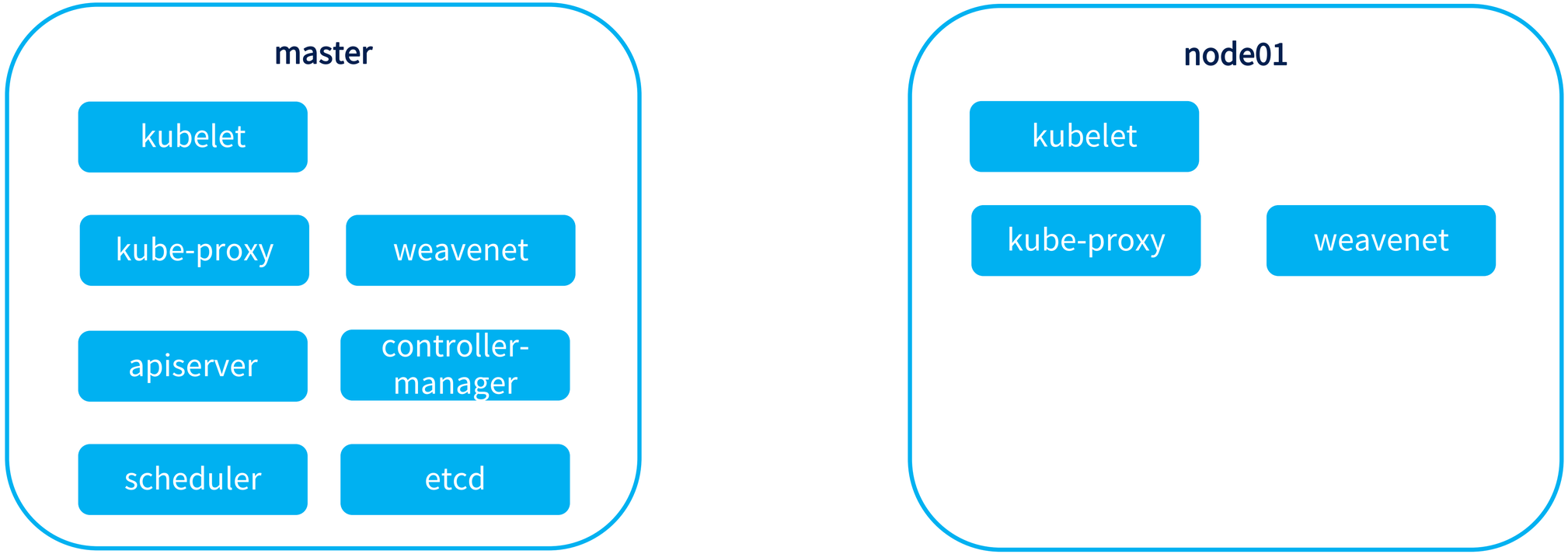
La ventaja es que nos permite explorar el Control Plane simplemente mediante kubectl.

```
kubectl --namespace kube-system get pod
```

Todos los componentes del control plane están desplegados en pods dentro del namespace kube-system.



Esquema del entorno de prueba

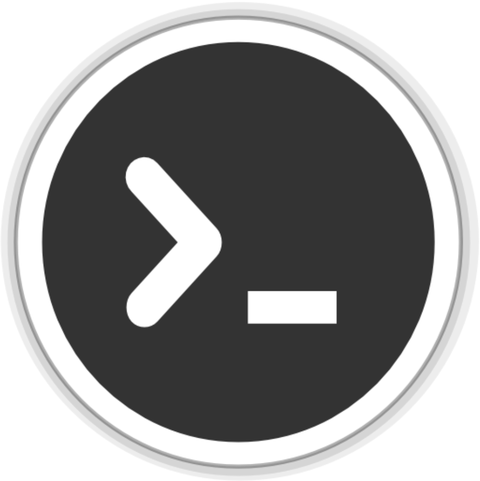


Exploramos estos componentes

Kubelet es el único componente que no puede correr dentro de un pod (ya que es el encargado de levantar propiamente los pods).

Para visualizar kubelet puedes ejecutar:

```
systemctl status kubelet
```

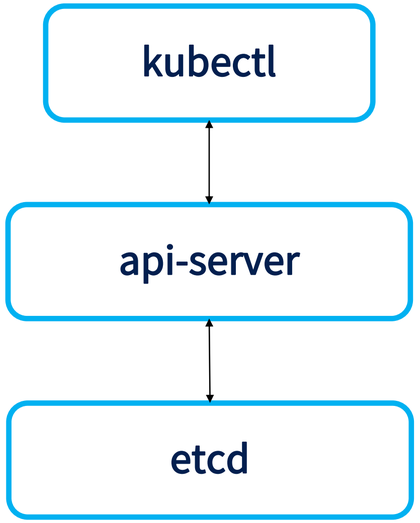


Flujo de un Pod en Kubernetes

1. Kubectl le envía nuestro comando al api-server

Por ejemplo, en todo el curso hemos ejecutado un comando para crear un pod, de nombre nginx, con la imagen de Docker nginx, y con el puerto 8080 abierto.

El api-server recibe la definición del pod, inmediatamente la almacena en la base de datos etcd, y comienza a trabajar para poder mandar ese pod a un nodo.



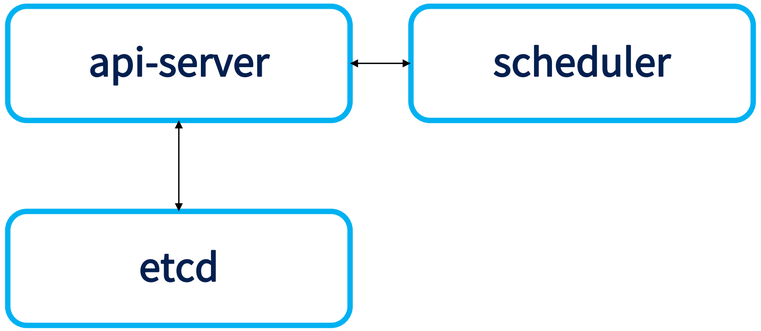
Flujo de un Pod en Kubernetes

2. El api-server se coordina con el scheduler

El scheduler es el encargado de analizar el estado de los nodos del clúster y determinar cuál de ellos puede ejecutar el pod.

Una vez ha elegido un nodo para nuestro pod (siempre que haya alguno disponible), envía la información de vuelta.

El api-server actualiza el Pod para asignarlo a un nodo en concreto. Cada actualización se almacena inmediatamente en etcd.



Flujo de un Pod en Kubernetes

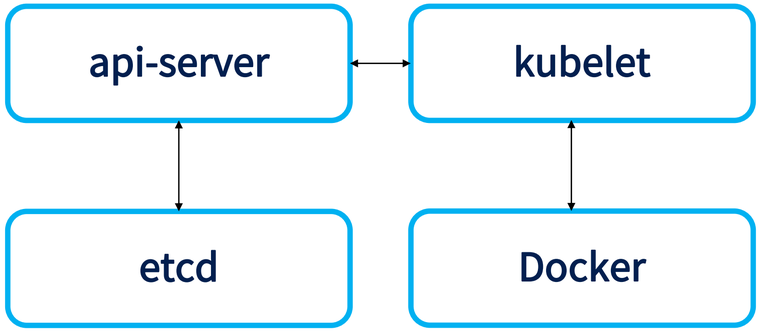
3. El api-server se coordina con kubelet

El api-server envía las órdenes al kubelet del nodo elegido para que ejecute ese pod en su nodo.

4. kubelet levanta los contenedores usando Docker

Kubelet se comunica con el proceso de Docker en su nodo para que ejecute los contenedores que componen el Pod, con los parámetros requeridos.

En todo momento kubelet envía información actualizada al api-server, y este almacena cada cambio en etcd.



Flujo de un Pod en Kubernetes

Durante todo este proceso podemos conocer el estado de nuestro Pod con kubectl get pod

Pasará por los diferentes estados:

1. **Pending**: api-server ha procesado la petición pero el scheduler todavía no ha asignado un nodo
2. **ContainerCreating**: El pod está asignado a un nodo pero Kubelet y Docker aún no han terminado de levantarlo (normalmente se encuentra descargando la imagen de Docker, montando volúmenes, etc.)
3. **Running**: El contenedor ya está ejecutándose correctamente

Instalación on-premise

Para desplegar un clúster de Kubernetes on-premise sólo hace falta levantar todas las piezas que hemos visto en este capítulo, hacerlo en alta disponibilidad y configurarlas adecuadamente.

Estas piezas puedes levantarlas en los nodos utilizando cualquier sistema de administración de procesos como SystemD o Supervisor. Cada una de las piezas son binarios que se ejecutan como cualquier otro proceso en Linux.

También es posible levantarlas dentro de contenedores o incluso de pods como se hace en el entorno de prueba. La única pieza que es necesario que no esté en contenedores es el propio Kubelet. Todas las demás pueden ser levantadas automáticamente por Kubelet, dentro de pods, o simplemente ejecutarlas en Docker. Existen imágenes de Docker oficiales de todas ellas.

Distribuciones on-premise

Aunque es posible hacerlo así, y de hecho te dará un mayor control de la configuración de tu clúster, al ser Kubernetes una herramienta de estándares, existen multitud de herramientas que automatizan la creación de clústeres, prácticamente con cualquier característica que necesites.

- **kubeadm**: Es la herramienta oficial y te permitirá levantar y mantener un clúster en alta disponibilidad de forma sencilla. Ha mejorado mucho en las últimas versiones.
- **Rancher**: Una herramienta visual para levantar uno o varios clústeres.
- **OpenShift**: Un distribución modificada de Kubernetes que se ha hecho muy popular

Hay muchísimas más cada una adaptada a unas necesidades o a unos proveedores cloud o sistemas operativos.

Distribuciones en local

Para comenzar a trabajar con Kubernetes no es necesario un clúster en producción y alta disponibilidad. Existen distribuciones mucho más sencillas que te permiten tener un clúster en local o en cualquier host con un sólo comando:

- **Minikube**: La solución oficial, levanta una máquina virtual como nodo del clúster
- **MicroK8S**: Una de mis preferidas por su sencillez
- **K3S**: La más ligera y sencilla de todas.

- **Docker for Windows and Mac**: Si utilizas estos sistemas operativos, lo mejor es simplemente activar el clúster de Kubernetes que viene por defecto en la instalación de Docker. Tendrás uno en cuestión de segundos.

¡Fin del módulo!

En este módulo hemos aprendido:

- La arquitectura de Kubernetes, todas las piezas que lo componen y la función de cada una de ellas.
- Cómo se reparten las responsabilidades los componentes del control plane.
- Qué es lo que define a un worker de Kubernetes, y cómo se comunica con el control plane y con Docker para ejecutar los contenedores.
- Hemos visto todas estas piezas en acción en el entorno de pruebas.
- Cómo alcanzar los requisitos de red que nos pide la plataforma para funcionar.
- Muchas distribuciones de Kubernetes, tanto para producción como para entornos locales o de pruebas.

Extras

Componentes Kubernetes
Workers
API Server
Scheduler
Controller Manager
Kubelet

¡Gracias!